

Minimização do Flowtime no Open Shop Problem

Desafios e Soluções Computacionais

Explorando métodos avançados para otimizar a produção industrial. Minimizando o tempo total de processamento em ambientes de produção flexíveis.

06 DE MAIO DE 2026

Introdução ao Open Shop Problem

Uma análise sobre o desafio de otimizar sequências de produção em ambientes com alta flexibilidade de processamento.

DEFINIÇÃO DO PROBLEMA

O escalonamento envolve ' n ' tarefas (jobs) a serem processadas em ' m ' máquinas distintas, onde cada job possui ' m ' operações.



Flexibilidade na Ordem

A principal característica é a flexibilidade: a sequência de operações de um job nas máquinas não é fixa, permitindo reordenamento para otimização.



Poppins

Restrições Cruciais

Cada máquina processa uma operação por vez, e cada job pode ser alocado em apenas uma máquina simultaneamente.

FLOWTIME

EXPLICAÇÃO DO FLOWTIME COMO MÉTRICA DE DESEMPENHO



Definição Central

O Flowtime mede a fluidez global do sistema, representando a soma dos tempos de conclusão de todos os trabalhos (jobs).



Objetivo na Otimização

A minimização do Flowtime é o objetivo principal, visando reduzir o tempo total que os trabalhos permanecem no sistema.



Estratégia Heurística

A regra SPT (Shortest Processing Time) é destacada como a melhor heurística simples para otimizar o Flowtime (Bräsel et al., 2008).



Prioridade Algorítmica

Em otimizações avançadas, o Flowtime global é priorizado, mesmo que isso exija sacrifícios em métricas secundárias como o Makespan.



Impacto no Desempenho

Um Flowtime otimizado garante que a maioria dos trabalhos flua livremente, resultando em maior eficiência e menor tempo de resposta geral.

Modelagem matemática



Grafos Direcionados Acíclicos (DAG)

Estrutura fundamental para representar operações e suas precedências. A aciclicidade garante a validade da sequência de tarefas, modelando o Open Shop Problem.



Caminhada Topológica

Consiste em movimentos (Swap, Insertion) aplicados em arestas disjuntivas do grafo para encontrar rearranjos que minimizem o Flowtime total.



Validação Topológica

O algoritmo de Kahn é empregado para validar a aciclicidade após cada alteração, permitindo a verificação paralela de múltiplas configurações de grafos.



Impacto no Fluxo

A análise topológica revela como a manipulação do DAG pode deslocar gargalos e otimizar caminhos críticos, resultando em uma fluidez global superior.

Implementação do Algoritmo de Kahn



Validação de Ciclos em Grafos Massivos

O algoritmo de Kahn é essencial para a validação topológica, crucial na detecção de ciclos em grafos extensos, garantindo a viabilidade das sequências no Open Shop Problem.

- **Cálculo de Dependências (Indegree):** O algoritmo inicia mapeando o grau de entrada (indegree) de cada nó. Operações sem dependências prévias (grau zero) são inseridas em uma Fila (std::queue).
- **Processamento e Baixa:** À medida que um nó é processado, suas arestas de saída são removidas, reduzindo o indegree dos nós vizinhos. Se um vizinho zerar, ele entra na Fila.
- **O Veredito (Detecção de Ciclo):** Se, ao esvaziar a fila, a contagem de nós visitados for menor que o número total de vértices do grafo, um ciclo foi detectado. Isso invalida o movimento, provando que o escalonamento é fisicamente impossível.

Fundamentação Teórica



Heurística SPT (Shortest Processing Time)

- Regra construtiva fundamental para a geração da baseline.
- Prioriza a execução de jobs com o menor tempo de processamento para otimizar o Flowtime.



Fundamentação Científica: Bräsel et al. (2008)

Este trabalho aponta a Heurística SPT como a regra mais eficaz entre as heurísticas simples para a minimização do Flowtime em problemas Open Shop.

Desafios em Matrizes Quadradas Densas ($n=m$)

- Prevista queda de qualidade e desempenho da heurística SPT em instâncias onde jobs (n) = máquinas (m).
- Esses cenários densos representam um gargalo, demandando métodos de otimização mais avançados.

Implementação da Baseline



Heurística SPT

Utilizada como regra construtiva inicial. É reconhecida como a heurística simples mais eficaz para otimizar o Flowtime no problema Open Shop.

(Bräsel et al., 2008)



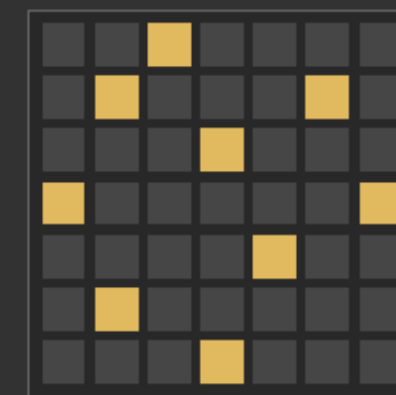
Desempenho Inicial

Para configurações de pequena escala, como matrizes 4x4, esta baseline provê soluções de forma rápida e com qualidade satisfatória.



Desafios em Escala

A abordagem SPT enfrenta uma queda de qualidade em matrizes densas, onde métodos sequenciais estagnam devido ao elevado conflito de recursos.



(Andresen, 2008)

Flowtime Baseline

Instancias	Flowtime Baseline
ta01Osp	1043
ta16Osp	2482
ta32Osp	9151
ta41Osp	22025
ta51Osp	39392

Limitações da Busca Local



Estagnação em Ótimos Locais

A "visão curta" da busca local tradicional impede a fuga de regiões onde melhorias imediatas não são evidentes, aprisionando o algoritmo.



Desempenho em Instâncias Críticas

Casos como `ta05osp` demonstram estagnação completa, resultando em 0% de melhoria (Flowtime: 1543 → 1543).



Insuficiência de Movimentos

As operações de Swap e Insertion, apesar de fundamentais, falham em superar barreiras de otimização e explorar novas regiões do espaço de busca.



Impossibilidade de Piora

A incapacidade de aceitar pioras temporárias impede o algoritmo de descer de "picos" locais para encontrar soluções globalmente superiores.

Flowtime Bloca

Problema de “melhor lugar”

Instancias	Melhoria Flowtime
ta05Osp	0%

Estratégias de Perturbação



Mecanismos de Shaking no VNS (multi start)

O Shaking evita a estagnação em ótimos locais, introduzindo perturbações controladas para impulsionar a busca para novas vizinhanças quando o progresso cessa.



Intensidade e Frequência

A intensidade é modulada para explorar diversas regiões, enquanto a frequência adaptativa, acionada pela falta de progresso, equilibra diversificação e eficiência.



Exploração vs. Exploração

O VNS equilibra a exploração intensiva da busca local com a exploração de novas áreas pelo Shaking, permitindo encontrar soluções globais mais robustas.

Problema métodos sequencias

Instancias	Melhoria Flowtime
ta031Osp	0%

O Limite do VNS: O mecanismo de Shaking esbarra no alto conflito de recursos de instâncias quadradas densas.

O Muro de Andresen (2008): A literatura documenta que métodos sequenciais estagnam em matrizes densas ($n=m$) devido a ciclos de repetição.

A Recomendação de Andresen

Hibridização e Vizinhança

A Necessidade Híbrida

A literatura é clara ao afirmar que a eficiência nesses cenários exige hibridização. É preciso unir a força de diferentes algoritmos para balancear a busca intensiva com a fuga de armadilhas matemáticas.

Exploração Ampla de Vizinhança:

Andresen recomenda não se limitar a vizinhos imediatos. Para achar rotas mais rápidas, o algoritmo precisa saltar para "vizinhanças distantes" no grafo e avaliar múltiplas estruturas topológicas simultaneamente.

Decisão

Baseado nesta recomendação, abandonamos a busca simples e desenhamos um motor que mescla a agressividade do VNS (Shaking) com a memória restritiva da Busca Tabu

Meta-heurística Híbrida

Vns + Lista Tabu



VNS (Variable Neighborhood Search)

Uma estratégia robusta para superar os "ótimos locais" inerentes à busca simples, explorando sistematicamente diferentes estruturas de vizinhança para encontrar soluções superiores.



Mecanismo de Shaking

Quando a otimização local estagna, o VNS emprega o Shaking para perturbar a solução atual, explorando novas vizinhanças e garantindo a fuga de regiões subótimas.



Integração com Busca Tabu

Para evitar ciclos e guiar a busca, uma lista Tabu memoriza movimentos recentes. Essa memória proíbe ações específicas, forçando o algoritmo a descobrir rotas alternativas e a aprofundar a busca.

Implementação da Busca Tabu



Mecanismo de Lista Tabu

Arestas de movimentação recente (ex: inversão X para Y) são inseridas numa lista de memória para proibir reversões imediatas.



Penalidades Temporárias

Uma "penalidade de turnos" é imposta, proibindo a reversão de um movimento por um número definido de iterações.



Exploração Alternativa

Esta restrição força o algoritmo a buscar rotas inexploradas, evitando ciclos e a estagnação em ótimos locais.

Paralelismo



12 Threads em C++

Utilização de `std::thread` para processamento massivo e concorrente.



Validação Simultânea

Verificação topológica concorrente de dezenas de grafos, acelerando a avaliação.



Kahn Paralelo

Execução do algoritmo para detecção de ciclos em paralelo, otimizando a validação.



Uso Máximo de CPU

Aproveitamento total dos 12 núcleos lógicos do processador para performance máxima.



Superação Sequencial

Elimina gargalos de métodos sequenciais na exploração de vizinhanças.

Análise de Desempenho

Instancias	Flowtime Baseline	Flowtime Otimizado	Melhoria Flowtime
ta16Osp	2482	1924	-22.48%
ta26Osp	5305	3989	-24.81%
ta28Osp	4528	3584	-20.85%
ta32Osp	9151	7922	-13.43%
ta51Osp	39392	33272	-15.54%

Análise de Desempenho

O motor híbrido superou a heurística SPT (Baseline) em 100% das faixas de tamanho analisadas, a arquitetura atingiu seu ápice de otimização operacional nas matrizes 7x7 reduzindo o tempo médio em 17,00%, resiliência no Gargalo: Mesmo nas instâncias massivas (20x20), onde o conflito de recursos ($n=m$) atinge o limite e a margem de manobra é mínima, o algoritmo garantiu uma redução sólida de quase 7,5%.

Instancias	4x4	5x5	7x7	10x10	15x15	20x20
Melhoria	-8,73%	-12,91%	-17,00%	-10,55%	-7,79%	-7,49%

"A análise comparativa por tamanho de instância valida a capacidade da abordagem híbrida em aprimorar a fluidez operacional."

Caso de Estudo: ta28

Análise do Paradoxo dos Saltos

📍 Caminho Critico

12 → 14

⌚ Makespan

910 ↓ 694

Estratégia "Via Expressa"

- 💡 **Análise Detalhada:** Em grafos ponderados, um caminho com mais nós não implica necessariamente um tempo maior.
- 👤 **Redirecionamento VNS:** A heurística VNS redireciona tarefas por máquinas ociosas, criando uma "via expressa".
- 📋 **Desvio de Gargalos:** Embora aumente o número de "saltos", a estratégia desvia de gargalos críticos, reduzindo o tempo de espera geral.

Caso de Estudo: ta60



Estratégia de Sacrifício do Gargalo

O algoritmo sobrecarrega intencionalmente o caminho de um único job, priorizando a fluidez global (Flowtime) em detrimento do Makespan.



Priorização do Job 4 para Liberar Fluxo

No caso ta60, o gargalo foi movido do Job 20 para o Job 4. Ao reter os conflitos no Job 4, os outros 19 jobs fluem livremente.



Impacto no Flowtime Global

Resulta na derrubada da soma total do Flowtime, otimizando a eficiência geral de todo o processo produtivo.



Otimização Colateral

Benefícios Adicionais na Minimização do Flowtime



Melhorias em Instâncias Menores

Em casos de até 10x10, a organização das "vias expressas" gerou benefícios adicionais e inesperados, otimizando também o Makespan.



Reduções Significativas

Foram observadas quedas de até **21%** no Makespan, com um impacto notável na instância de teste TA10.



O Efeito "Vias Expressas"

A criação de rotas alternativas para jobs melhorou o tempo total de produção em cenários de menor complexidade, além do objetivo principal.



Trade-off Flowtime vs Makespan



Impacto Extremo

Nas matrizes de maior porte (15x15 e 20x20), observamos um comportamento rigorosamente planejado e controlado.



Piora Controlada

O Makespan apresentou pioras médias mínimas, variando de **+0.07% a +0.70%**,



Prioridade Estratégica

Este sacrifício marginal e deliberado no Makespan é uma decisão consciente para priorizar a fluidez do processo.



Benefício Global

Assegura a excelência sistêmica e a fluidez, otimizando o Flowtime geral para ganhos mais significativos na operação.

Trade-off Makespan

Tamanho	Makespan (Pioras em valores positivos)
4x4	-10.70%
5x5	-16.50%
7x7	-15.36%
10x10	-7.16%
15x15	+0.07%
20x20	+0.70%

Conclusões



Síntese dos Resultados Chave

- A meta-heurística híbrida (VNS, Busca Tabu, paralelismo) demonstrou superioridade na otimização do Flowtime, superando armadilhas de ótimos locais.
- Alcançou uma melhoria média de 17% no Flowtime em instâncias 7x7, destacando a eficácia de estratégias como "Sacrifício do Gargalo".



Trade-off Consciente de Métricas

- O algoritmo prioriza a fluidez global (Flowtime), aceitando um pequeno comprometimento do Makespan (+0.07% a +0.70%) para ganhos significativos no fluxo.
- Em instâncias simples, a otimização do Flowtime também reduziu o Makespan (até 21%).



Direções Futuras para Pesquisa

- Investigar outras funções objetivo (minimização do Makespan, prazos de entrega).
- Aplicação e validação em cenários dinâmicos e instâncias industriais de maior porte.
- Explorar a integração com aprendizado de máquina para adaptação heurística.

Referências Bibliográficas

Fundamentação Teórica

- **Bräsel, H. et al. (2008).** *Constructive Heuristic Algorithms for the Open Shop Problem. European Journal of Operational Research.*
- **Andresen, M. et al. (2008).** *Simulated Annealing and Genetic Algorithms for the Open Shop Scheduling Problem.*

Base de Dados e Benchmarks

-  As instâncias de teste incluem séries padronizadas da literatura, como '**ta**' (e.g., **ta05**, **ta28**, **ta60**), amplamente reconhecidas em problemas de escalonamento.

Links



Planilha

<https://docs.google.com/spreadsheets/d/1OCVjgphW9PZcB0Gh2DISvuyNJWrwXtZXMzWTLuSP9ko/edit?usp=sharing>



GitHub

<https://github.com/GabrielDavi7/OpenShop-Flowtime>

Obrigado